

Rapport d'audit de performance — Simulation de colonie de fourmis

Sup de Vinci — RNCP Bloc 4 · Optimisations & Performances Backend Auteur : Leo Bessin ·
Langage : Go · Session E42

État : en cours. Sections 1, 2 et 5 renseignées. Section 3 : v1 `flatgrid` mesuré et rédigé ; v4/F1/v6 restent à construire (chemin retenu, §3). Section 4 : F1 planifié, pas encore construit. Une entrée `docs/journal/` par levier.

Les tableaux générés automatiquement sont dans `docs/report/generated-tables.md` (`make report`). Ne pas les recopier à la main : les régénérer.

0. Objet et périmètre

Simulation discrète d'une colonie de fourmis sur grille : fourmis, nourriture, murs, et deux champs de phéromones (piste *food* et piste *home*). Le programme prend une configuration JSON en entrée et produit un résultat déterministe en sortie, ce qui rend chaque version mesurable sans interaction.

Invariant de correction. Même configuration + même graine $\Rightarrow$ `StateChecksum` identique, sur tous les moteurs. Toute optimisation qui change ce checksum n'est pas une optimisation : c'est une régression fonctionnelle, et sa vitesse n'est pas reportée.

Baseline délibérément naïve. Le moteur v0 (`internal/engine/naive`) utilise des `map[string]` avec clés `fmt.Sprintf`, un slice de pointeurs et des allocations par tick. Ce choix est assumé et documenté : il fournit une marge de progression réelle, et chaque structure naïve prépare un levier précis du cours. Ce qui est préservé dès v0 — PRNG par fourmi, arithmétique entière en virgule fixe, double tampon, phases séparées — l'est parce que ce sont les propriétés qui rendront les optimisations ultérieures *prouvablement* équivalentes.

1. Environnement & métrologie (§1 — 3 pts)

→ source : [docs/journal/00-metrology.md](https://docs.vinci.fr/journal/00-metrology.md)

1.1 Banc d'essai

Élément	Valeur
CPU	Apple M4 Pro — 12 cœurs physiques / 12 threads (pas de SMT)
L1	128 Ko (icache) + 64 Ko (dcache), par cœur
L2	4096 Ko
L3	non exposé (Apple Silicon utilise un SLC système, non instrumentable via <code>sysctl</code>)
RAM	24 Go
OS	macOS 27.0 (build 26A428)
Runtime	go1.26.4 darwin/arm64 · CGO_ENABLED=0

Capture automatique et horodatée : `make env` → `docs/env/` (dernière capture : [docs/env/2026-09-22_164722.txt](#)).

1.2 Protocole

Instrument	Question à laquelle il répond
<code>go test -bench</code>	Quelle instruction / allocation a diminué ? (<code>ns/op</code> , <code>B/op</code> , <code>allocs/op</code> , courbe <code>-cpu</code>)
<code>hyperfine</code>	Qu'attend réellement l'opérateur ? (process complet, moyenne / médiane / écart-type)

1.3 Dimensionnement des scénarios vis-à-vis des caches

Une fois la grille aplatie en `[]uint32` (étage v1), une grille coûte `W × H × 4` octets :

Scénario	Grille	Par grille	Niveau
<code>small</code>	64×64	16 Ko	L1/L2
<code>medium</code>	128×128	64 Ko	L2
<code>large</code>	256×256	256 Ko	4 tampons dépassent L2, tiennent en L3

Le palier `medium` → `large` est l'endroit où le travail de localité doit devenir visible.

2. Diagnostic matériel & profilage (§2 — 5 pts)

→ source : [docs/journal/01-profiling.md](#) → artefacts : `bench/profiles/naive.cpu.pprof` , `naive.mem.pprof` , `*.top.txt`

2.1 Hot Path identifié

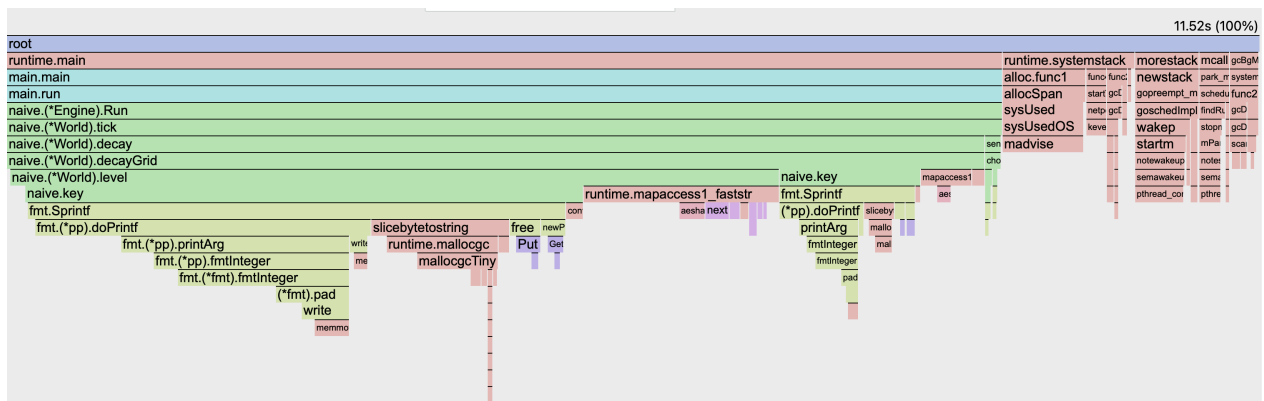
Profil capturé sur le scénario `medium` (128×128, 400 fourmis, 400 ticks) :

```
./bin/antsim.exe -quiet -config internal/config/scenarios/medium.json \
  -engine naive -cpuprofile bench/profiles/naive.cpu.pprof \
  -memprofile bench/profiles/naive.mem.pprof
go tool pprof -top -nodecount=25 bench/profiles/naive.cpu.pprof
go tool pprof -top -nodecount=25 -sample_index=alloc_objects
  bench/profiles/naive.mem.pprof
```

Le goulot est l'**adressage des cellules**, pas la logique de simulation.

- `fmt.Sprintf` : **55,13 % du CPU cumulé, 99,84 % des allocations**
- `runtime.mapaccess1_faststr` : 19,74 % cumulé
- `naive.(*World).level` (lecture des 4 voisins) : **59,23 % du CPU cumulé, 78,44 % des allocations**
- 97,79 % des allocations traversent `decayGrid`

Flamegraph annoté (`go tool pprof -http=:8080 bench/profiles/naive.cpu.pprof`, capture sur `-repeat 3` — d'où les 11,52 s en racine, contre 3,90 s pour le run unique cité ci-dessus ; la forme de l'arbre est identique) :



La largeur de chaque case est proportionnelle au temps CPU. La pile `naive.(*World).decayGrid` → `level` → `key` → `fmt.Sprintf` → `doPrintf` → `printArg` → `fmtInteger` occupe à elle seule l'essentiel de la largeur de l'image — c'est la lecture visuelle du même chiffre que le `-top` (55,13 %). À sa droite, `runtime.mapaccess1_faststr` (hachage + sondage de la map) forme une pile sœur plus étroite mais toujours large. Le tiers droit de l'image (`runtime.systemstack`, `allocSpan`, `madvice`) est le coût du GC déclenché par ces mêmes allocations — une conséquence du même problème, pas une cause séparée.

Ligne exacte, `pprof -list` (pas seulement le nom de la fonction) :

```
go tool pprof -list 'naive\key$' -trim_path=antcolony/
  bench/profiles/naive.cpu.pprof
```

```
ROUTINE ===== antcolony/internal/engine/naive.key in internal/engi
0      2.20s (flat, cum) 56.41% of Total
.      2.20s   105:func key(x, y int64) string { return fmt.Sprintf("%d,%d"
```

Une seule ligne de code — `naive.go:105` — consomme 56,41 % du temps CPU de tout le programme. `-trim_path=antcolony/` est requis : le binaire est compilé avec `-trimpath` (§5.1),

donc `pprof` cherche par défaut `antcolony/internal/engine/naive/naive.go` sur le disque au lieu du chemin relatif réel.

Vérification arithmétique : 128×128 cellules $\times$ 5 adressages $\times$ 2 grilles $\times$ 400 ticks $\approx$ **65,5 M** appels à `key()`, à comparer aux **66,3 M** allocations mesurées (`Engine/medium/naive` ci-dessous). Le coût dominant est donc proportionnel à la **surface de la grille**, pas au nombre de fourmis (`chooseMove` : 1,03 % du CPU, 1,68 % des allocations — vérifié à la ligne près avec `pprof -list naive.\(*World\).chooseMove`).

Artefacts : [naive.cpu.top.txt](#), [naive.mem.top.txt](#), [image.png](#) (flamegraph).

2.2 Micro-benchmarks — référence v0 (à comparer à chaque étage futur)

```
go test -run '^$' -bench '^Benchmark(Key|GridLookup|Rng)' -benchmem \
  -benchtime 3s -count 8 ./bench/...
```

Médianes `benchstat`, écart-type entre parenthèses :

Benchmark	Rôle	ns/op	B/op	allocs/op
<code>KeySprintf</code>	clé <code>fmt.Sprintf("%d,%d", x, y)</code>	45,29 (± 3 %)	6	1
<code>KeyStrconv</code>	clé via <code>strconv</code>	20,45 (± 1 %)	7	1
<code>KeyFlatIndex</code>	index <code>y*W+x</code>	0,3161 (± 2 %)	0	0
<code>GridLookupMap</code> (128 $\times$ 128)	lecture grille par clé map	56,00 (± 1 %)	6	1
<code>GridLookupFlat</code> (128 $\times$ 128)	lecture grille indexée	0,3245 (± 1 %)	0	0
<code>RngXoshiro</code> (Next)	tirage PRNG maison	2,694 (± 1 %)	0	0
<code>RngXoshiroIntn</code>	tirage borné maison	2,889 (± 0 %)	0	0
<code>RngMathRandLocal</code>	<code>math/rand</code> , source locale	2,830 (± 0 %)	0	0
<code>RngMathRandGlobal</code>	<code>math/rand</code> , source globale	5,864 (± 1 %)	0	0

Rapport map / indexé mesuré : 172,6 $\times$ (56,00 / 0,3245). Décomposition : 20,4 ns de construction de chaîne (`strconv`), 24,8 ns de surcoût `fmt` (`Sprintf` – `Strconv`), $\approx$ 10,7 ns d'accès map pur (`GridLookupMap` – `KeySprintf`), contre 0,32 ns pour un accès indexé. Même le meilleur cas avec clé chaîne (`strconv` + accès map pur : 20,4 + 10,7 = 31,2 ns) resterait à **96 $\times$ l'indexation** : il faut supprimer la chaîne, pas l'optimiser.

Métrologie. Aux réglages initiaux (1s, `-count 6`) ces mesures affichaient ± 30 -43 % d'écart et `GridLookupMap` ressortait *plus rapide* que le `KeySprintf` qu'il contient — un ordre impossible, donc du bruit. Les réglages par défaut ont été portés à 3s / `-count 8`, ramenant l'écart à ± 1 -3 %. Un benchmark dont la variance dépasse l'effet mesuré ne mesure rien.

2.3 Borne d'Amdahl

`Sprintf` (55,13 %) + `mapaccess1_faststr` (19,74 %) $\approx$ **74,9 %** du temps CPU cumulé $\Rightarrow$ le gain maximal atteignable en les supprimant est borné à $\approx$ **4,0 $\times$** sur cette fraction ($1 / (1 - 0,749)$),

avant apparition du goulot suivant (bande passante mémoire de `decayGrid` , ou pression GC).

Vérifié en v1 (§3) : le gain réel est $\approx 106\times$, pas $\approx 4\times$. La borne ci-dessus ne comptait que la fraction CPU d'un seul échantillon ; elle traitait le reste du profil comme un temps fixe alors qu'une bonne partie (`mallocgcTiny` , `madvise` , `sync.Pool.Get/Put`) était elle-même causée par les mêmes 66 M allocations. Une borne d'Amdahl sur un profil échantillonné est un minorant, pas une prédiction — détail et explication mécanique dans [docs/journal/04-flatgrid.md](https://docs.flatgrid.md) .

2.4 Utilisation CPU & RAM (compteurs matériels)

`pprof` mesure des **pourcentages de temps CPU**, pas l'utilisation réelle des cœurs ni la mémoire effectivement occupée par le process. Deux instruments distincts pour ces deux questions-là :

```
/usr/bin/time -l ./bin/antsim.exe -quiet -config
    internal/config/scenarios/medium.json -engine naive >/dev/null
/usr/bin/time -l ./bin/antsim.exe -quiet -config
    internal/config/scenarios/medium.json -engine flatgrid >/dev/null
```

Métrique	v0 naive	v1 flatgrid	Rapport
Temps réel (real)	3,94 s	0,04 s	98,5×
Temps CPU (user)	4,25 s	0,04 s	106×
Instructions retirées	93,06 G	1,28 G	72,5×
Cycles écoulés	18,47 G	0,19 G	96,0×
IPC (instructions/cycle)	5,04	6,67	—
Empreinte mémoire crête (peak memory footprint)	19,4 Mio	10,6 Mio	1,84×
RSS max (maximum resident set size)	21,5 Mio	12,6 Mio	1,71×

Deux lectures, pas une seule :

1. **CPU — corroboration indépendante du gain.** Le compteur matériel d'instructions (72,5×
- et de cycles (96,0×
- confirme le $\approx 106\times$ mesuré par `benchstat` et `hyperfine` (§5.3), par une voie complètement différente (compteurs du noyau, pas échantillonnage `pprof`). Fait notable : `user` (4,25 s) **dépasse** `real` (3,94 s) sur `naive` — plus d'un cœur a travaillé en moyenne, alors que le tick lui-même est strictement mono-thread (§3.2). C'est le ramasse-miettes concurrent de Go : les 66 M allocations/run forcent un marquage/balayage en tâche de fond sur des threads séparés. `flatgrid` n'a quasiment plus cet écart (`user` $\approx$ `real`) — encore une confirmation que la pression GC, pas la boucle de tick, consommait le cœur supplémentaire.
2. **RAM — ce n'est PAS le même signal que B/op .** B/op (§2.2, §5.3) mesure un **débit d'allocation cumulé sur tout le run** (397,8 Mio pour `naive` , $\div 46$ en `flatgrid`) — c'est du mémoire *churné*, alloué puis collecté en boucle. L'empreinte **crête** réelle du process ne bouge que d'un facteur **1,7-1,8×** (19,4 $\rightarrow$ 10,6 Mio). `naive` n'a jamais eu un problème de RAM au sens où le process "dépasserait" un budget — le problème était le débit d'allocation

(donc le CPU consommé à allouer/collecter), pas la quantité retenue en mémoire à un instant donné. Sur les 24 Go de cette machine (§1.1), les deux moteurs sont de toute façon anecdotiques : le sujet n'est jamais "est-ce que ça tient en RAM" mais "combien de cycles CPU le trafic mémoire consomme-t-il" — ce que la hiérarchie de cache (§1.3) mesure mieux que la RAM totale.

3. Journal d'optimisation

Chemin retenu. §3 exige 3 axes, §4 un échec constructif : 4 étages suffisent (`v1` → `v4` → `F1` → `v6`), chacun mesuré et expliqué en entier, plutôt qu'une liste longue et sous-documentée. `v2` / `v3` / `v5` restent en option, seulement si le chemin retenu est fini et chiffré avant la deadline (détail : [CLAUDE.md](#)).

Gabarit de rédaction — une entrée `docs/journal/0N-<étage>.md` par ligne du chemin retenu

Chaque entrée reprend ces 4 sous-sections, dans cet ordre, pour rester comparable d'un étage à l'autre :

1. **Optimisation appliquée** — l'hypothèse matérielle (quelle ligne du profil de l'étage précédent la justifie), le changement unique apporté, et le fichier touché (`internal/engine/<étage>/...`).
2. **Métriques de référence** — les chiffres de l'étage précédent, copiés tels quels (pas remesurés a posteriori), pour que le delta ne soit pas calculé contre une cible mouvante.
3. **Comparatif benchmark** — un tableau à deux colonnes (`vN-1` vs `vN`) sur les mêmes métriques que le tableau global (§5.3) : `ns/op` ou `sec/op`, `B/op`, `allocs/op`, et le % `benchstat` avec sa significativité.
4. **Commande** — la commande exacte relancée pour produire ces chiffres, copiable telle quelle (`go test -bench ...`, `benchstat ...`, `hyperfine ...`).

La ligne `v0` → `v1`, remplie — entrée complète : [docs/journal/04-flatgrid.md](#).

1. Optimisation appliquée. Hypothèse (§2.3) : `fmt.Sprintf` + `mapaccess1_faststr` ≈ 74,9 % du CPU cumulé, donc les remplacer par un index plat devrait au moins tripler la vitesse. Changement unique : les 8 `map[string]T` du `World` (`Walls`, `Food`, `PheroFood`, `PheroHome`, `depFood`, `depHome`, `nextFood`, `nextHome`) deviennent des `[]T` indexés `idx(x,y) = y*W+x`. Tout le reste — PRNG, virgule fixe, double tampon, `Ant` non alignée, `[]*Ant` — copié verbatim de `naive` (leviers de `v2/v3`). Fichiers : `internal/engine/flatgrid/{flatgrid,tick,output}.go`.

2. Métriques de référence (v0). `Engine/medium` : 4,029 s ± 1 %, 397,8 Mi, 66,30 M `allocs/op`. `TickRate` : 96,47 ticks/s. `Checksum medium` : `0xa6b0a8c6451d55e4`.

3. Comparatif benchmark (`-count 6`, médianes) :

Métrique	v0 naive	v1 flatgrid	Gain
Engine/medium sec/op	4,029 s ± 1 %	37,88 ms ± 1 %	≈ 106×
allocs/op	66 302 084	165 315	≈ 401×
B/op	417 160 573	8 990 112	≈ 46×
TickRate ticks/s	96,47	≈ 9 944	≈ 103×
Binaire complet (hyperfine)	4,060 s ± 0,096 s	43,2 ms ± 0,8 ms	94,07× ± 2,83
Checksum	0xa6b0a8c6451d55e4	0xa6b0a8c6451d55e4	identique

make test : PASS, checksum identique — l'invariant de correction tient. Le gain (≈106×) dépasse largement la borne d'Amdahl (§2.3) : voir le journal pour l'explication (la borne ignorait la pression GC causée par les mêmes allocations qu'elle ne comptait pas).

4. Commande.

```
go test -run '^$' -bench '^Benchmark(Engine|TickRate)' -benchmem \
    -benchtime 3x -count 6 ./bench/...
hyperfine --warmup 3 --runs 10 -L engine naive,flatgrid \
    "./bin/antsim.exe -quiet -config internal/config/scenarios/medium.json -
    engine {engine}"
```

3.1 Mémoire & localité de cache

- ☒ v1 flatgrid — `map[string]T → []T, idx = y*W+x`. ≈106× sur Engine/medium, allocs/op ÷401, checksum identique. Détail : [docs/journal/04-flatgrid.md](https://docs.journal/04-flatgrid.md).

3.2 Concurrency & scalabilité CPU

- ☐ v4 parallel — worker pool dimensionné aux 12 cœurs **physiques** (§1.1), phases A et C découpées en bandes de lignes — construit sur v1
- ☐ (optionnel) v5 tuned — compteurs atomiques, arrêt précoce, taille de bande calée sur L2

Référence — reproduction :

```
go test -run '^$' -bench BenchmarkScaling -benchmem \
    -benchtime 3x -count 6 -cpu 1,2,4,8,12 ./bench/...
```

GOMAXPROCS	sec/op	B/op	allocs/op
1	566,4m ± 2 %	41,47 Mi	8,266 M
2	512,6m ± 1 %	41,47 Mi	8,266 M
4	511,9m ± 1 %	41,47 Mi	8,266 M
8	513,6m ± 2 %	41,48 Mi	8,266 M
12	505,3m ± 3 %	41,49 Mi	8,266 M

La courbe est **plate** (566 → 505 ms, moins de 12 % d'écart total, sans tendance monotone au-delà du bruit) : c'est la preuve formelle que v0 est mono-thread, quel que soit `GOMAXPROCS` . Source : [scaling_2026-09-22_164722.txt](https://github.com/golang/go/blob/master/src/runtime/trace/trace.go#L164722) .

3.3 I/O réseau & persistance

- ☐ v6 `codec` — gob / Protobuf vs JSON ; `sync.Pool` sur les snapshots
- ☐ SQLite : journal des exécutions, `EXPLAIN QUERY PLAN` , index sur `(config_hash, engine)`

4. Confrontation critique & « échec constructif » (§4 — 3 pts)

- ☐ F1 `failsharing` — **faux partage** : compteurs par worker non rembourrés dans une même ligne de 64 octets, puis le rembourrage `[64]byte` qui corrige. Publier les deux chiffres et l'explication mécanique.

Candidat retenu parce qu'il est mesurable, explicable en termes matériels, et qu'il porte directement sur le contenu « lignes de cache 64 octets » du cours.

5. Reproductibilité & synthèse (§5 — 4 pts)

5.1 Commande unique

```
bash run_benchmarks.sh
```

Étapes : `env` → `test` (**gate de correction, abandon si échec**) → `bench` + `benchstat` → `build` → `hyperfine` → `pprof` → `report` .

Aucun chiffre n'est publié depuis un arbre où `go test ./...` échoue.

5.2 Commandes

Tout ce qui a produit un chiffre cité dans ce rapport, en une seule table :

Action	make	Commande brute équivalente
Compiler / vérifier	—	<code>go vet ./...</code>
Gate de correction	<code>make test</code>	<code>go test ./...</code>
Test ciblé	—	<code>go test ./test/... -v -run 'NomDuTest'</code>
Spec machine	<code>make env</code>	capture <code>sysctl / sw_vers</code> → <code>docs/env/</code>
Benchmarks moteurs	<code>make bench</code>	<code>go test -run '^\$' -bench '^Benchmark(Engine TickRate Scaling)' -benchmem -benchtime 3x -count 6 ./bench/...</code>
Benchmarks micro	<code>make bench</code>	<code>go test -run '^\$' -bench '^Benchmark(Key GridLookup Rng)' -benchmem -benchtime 3s -count 8 ./bench/...</code>
Scalabilité	<code>make bench</code>	<code>go test -run '^\$' -bench BenchmarkScaling -cpu 1,2,4,8,12 -benchtime 3x -count 6 ./bench/...</code>
Comparaison statistique	<code>make bench</code>	<code>benchstat bench/results/baseline.txt bench/results/latest.txt</code>
Build	<code>make build</code>	<code>go build -trimpath -o bin/antsim.exe ./cmd/antsim</code>
Temps binaire complet	<code>make hyper</code>	<code>hyperfine --warmup 3 --runs 10 --export-markdown bench/results/hyperfine.md "./bin/antsim.exe -quiet -config internal/config/scenarios/medium.json -engine {engine}"</code>
Profils CPU + tas	<code>make profile</code>	<code>./bin/antsim.exe -quiet -config ... -engine naive -cpuprofile ... -memprofile ...</code>
Top du profil	<code>make profile</code>	<code>go tool pprof -top -nodecount=25 naive.cpu.pprof</code>
Ligne exacte du hot path	—	<code>go tool pprof -list 'naive\.key\$' -trim_path=antcolony/ naive.cpu.pprof</code>
Flamegraph interactif	—	<code>go tool pprof -http=:8080 naive.cpu.pprof</code>
Isoler par mot-clé	—	<code>go tool pprof -top -cum -focus=key naive.cpu.pprof</code>
CPU/RAM réels (compteurs matériels)	—	<code>/usr/bin/time -l ./bin/antsim.exe -quiet -config ... -engine {engine}</code>
Tableaux du rapport	<code>make report</code>	concatène <code>bench/results/</code> , <code>bench/profiles/</code> → <code>docs/report/generated-tables.md</code>
Exécution headless unique	<code>make run</code>	<code>./bin/antsim.exe -config internal/config/scenarios/medium.json -engine naive</code>
Interface web	<code>make web</code>	<code>go run ./cmd/antweb</code>

Action	make	Commande brute équivalente
Régénérer les golden (nouvelle définition seulement)	<code>make golden</code>	<code>UPDATE_GOLDEN=1 go test ./test/... -run TestGolden -v</code>
Tout, en une commande	<code>make all</code>	<code>bash run_benchmarks.sh</code>

5.3 Comparatif global des approches

Une ligne par étage, verdict inclus — pas seulement les chiffres. Colonnes vides = étage pas encore construit (voir chemin retenu, §3).

Étage	Approche	Engine/medium sec/op	Allocs/op	Concurrence (-cpu 1→12)	Conclusion
v0	<code>naive — map[string]uint32 + clé fmt.Sprintf, []*Ant</code>	4,029 s ± 1 %	66,30 M	plate, 566→505 ms (mono-thread confirmé)	Baseline. Goulot identifié : <code>key()</code> = 56,41 % du CPU sur une seule ligne (§2.1).
v1	<code>flatgrid — map[string]T → []T, idx=y*W+x (8 champs)</code>	37,88 ms ± 1 %	165 315	non testé (v4 découpe en bandes, pas cet étage)	Retenu : ≈106× sur Engine/medium , allocs ÷401, checksum identique à naive. Dépasse la borne Amdahl (≈4×) — voir journal : la borne ignorait la pression GC causée par les mêmes allocations. Hot path suivant : <code>decayGrid / level</code> (bande passante mémoire), comme prédit.
v4	<code>parallel — worker pool 12 cœurs, construit sur v1</code>	—	—	—	<i>Prochain étage : v1 est mesuré, la grille plate permet enfin un découpage en bandes propre.</i>
F1	<code>failsharing — faux partage puis rembourrage [64]byte</code>	—	—	—	<i>Échec constructif planifié (§4) : deux chiffres à publier, pas un.</i>
v6	<code>codec — gob/Protobuf vs JSON</code>	—	—	—	<i>À construire, indépendant du reste.</i>

Sources versionnées : [bench/results/baseline.txt](#) , [bench/results/benchstat.txt](#) ,
[bench/results/hyperfine.md](#) . `baseline.txt` est le v0 pinné sur **cette** machine (§1.1) — chaque

étage futur tourne `benchstat` `baseline.txt` `latest.txt` contre lui pour obtenir la colonne `Conclusion` (delta % + significativité), à recopier ici une fois disponible plutôt que de ressaisir les chiffres à la main.

6. Bonus — gouvernance IA (§6 — +2 pts)

→ [constitution.md](#)

Les quatre directives : rôle système strict (ingénieur contraint par des métriques physiques réelles) ; contraintes négatives explicites (`fmt.Sprintf` , maps, flottants, allocations et goroutines non bornées interdits sur le Hot Path) ; justification empirique obligatoire sous la forme « hypothèse d'impact matériel / commande de vérification » ; formulation compacte et impérative.